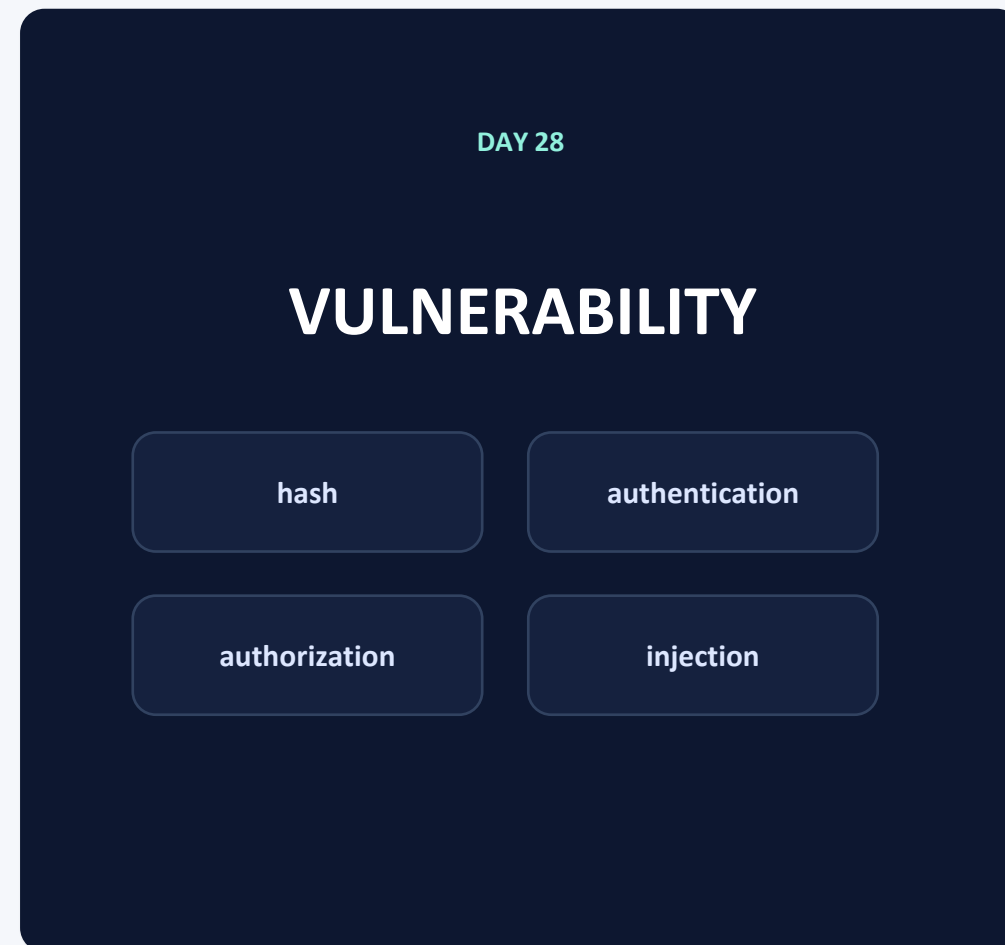


Безопасность приложений

Увидеть базовые риски веб-приложений: injection, XSS/CSRF, секреты, authentication vs authorization, hashing и принцип наименьших привилегий.



ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

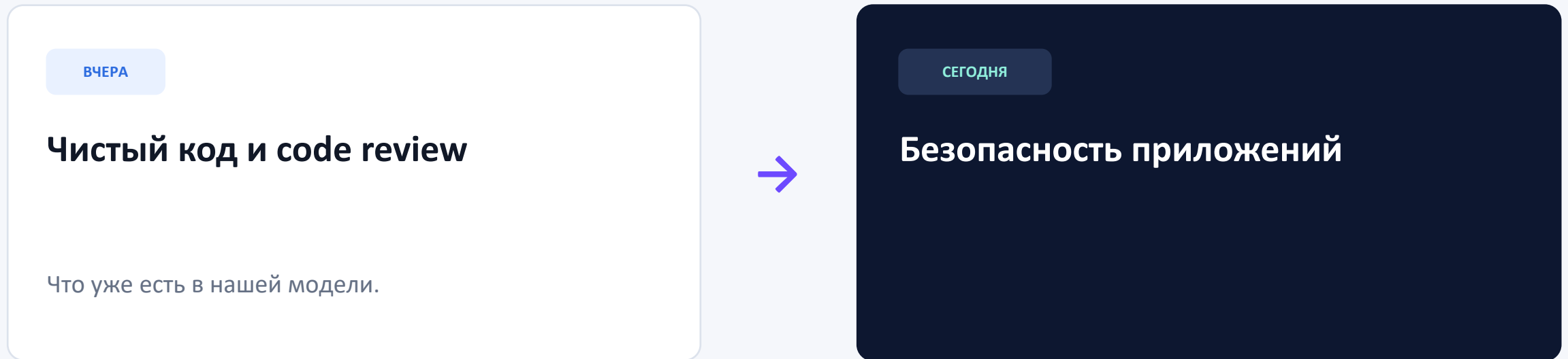
Конкретные действия вместо списка терминов.

- 01 Увидеть базовые риски веб-приложений: injection, XSS/CSRF, секреты, authentication vs authorization, hashing и принцип наименьших привилегий.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.



Вчера: Чистый код и code review. Сегодня мы добавляем следующий слой: безопасность приложений.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Работающее приложение может быть опасным, если доверяет пользовательскому вводу или выдаёт больше прав, чем нужно.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Безопасность — не один замок, а несколько слоёв: проверка личности, проверка права доступа, безопасная обработка данных и минимально необходимые ключи.

ПЕРЕВОД В IT

vulnerability

слабое место, которое можно использовать во вред

hash

одностороннее преобразование данных в фиксированное значение

authentication

проверка личности пользователя

Слой 1: vulnerability • hash • authentication

Сначала смысл, затем профессиональный термин.

**vulnerability**

слабое место, которое можно использовать во вред

**hash**

одностороннее преобразование данных в фиксированное значение

**authentication**

проверка личности пользователя

Слой 2: authorization • injection • secret

Сначала смысл, затем профессиональный термин.

authorization

проверка разрешения на конкретное действие

injection

внедрение пользовательского ввода в команду или запрос

secret

закрытое значение вроде ключа, токена или пароля

Слой 3: encryption • CSRF

Сначала смысл, затем профессиональный термин.

encryption

обратимое преобразование данных с секретным ключом

CSRF

атака с выполнением запроса от имени вошедшего пользователя

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: XSS • least privilege

Сначала смысл, затем профессиональный термин.

XSS

внедрение вредоносного скрипта в веб-страницу

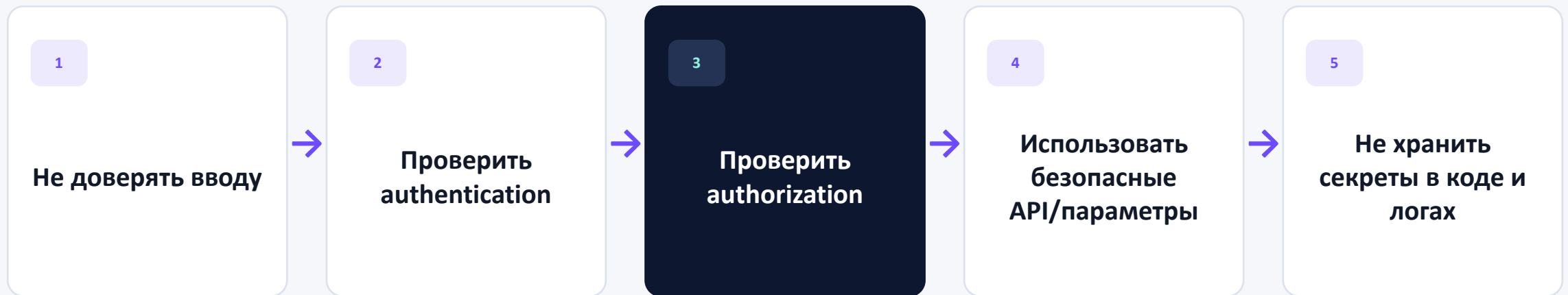
least privilege

выдача только минимально необходимых прав

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day28_step1

```
query = "SELECT id, email FROM users WHERE email = ?"  
params = (email,)   
cursor.execute(query, params)
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
query = "SELECT id, email FROM users  
WHERE email = ?"
```

Сохраняем значение или результат вычисления под понятным именем.

```
params = (email,)
```

Сохраняем значение или результат вычисления под понятным именем.

```
cursor.execute(query, params)
```

Python выполняет эту строку сверху вниз.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
query = "SELECT id, email FROM users WHERE email  
params = (email,)  
cursor.execute(query, params)
```

РЕЗУЛЬТАТ

Значение email передаётся
SQL-текста

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
import os
api_key = os.environ.get("API_KEY")
print(api_key is not None)
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

True, если секрет передан через окружение

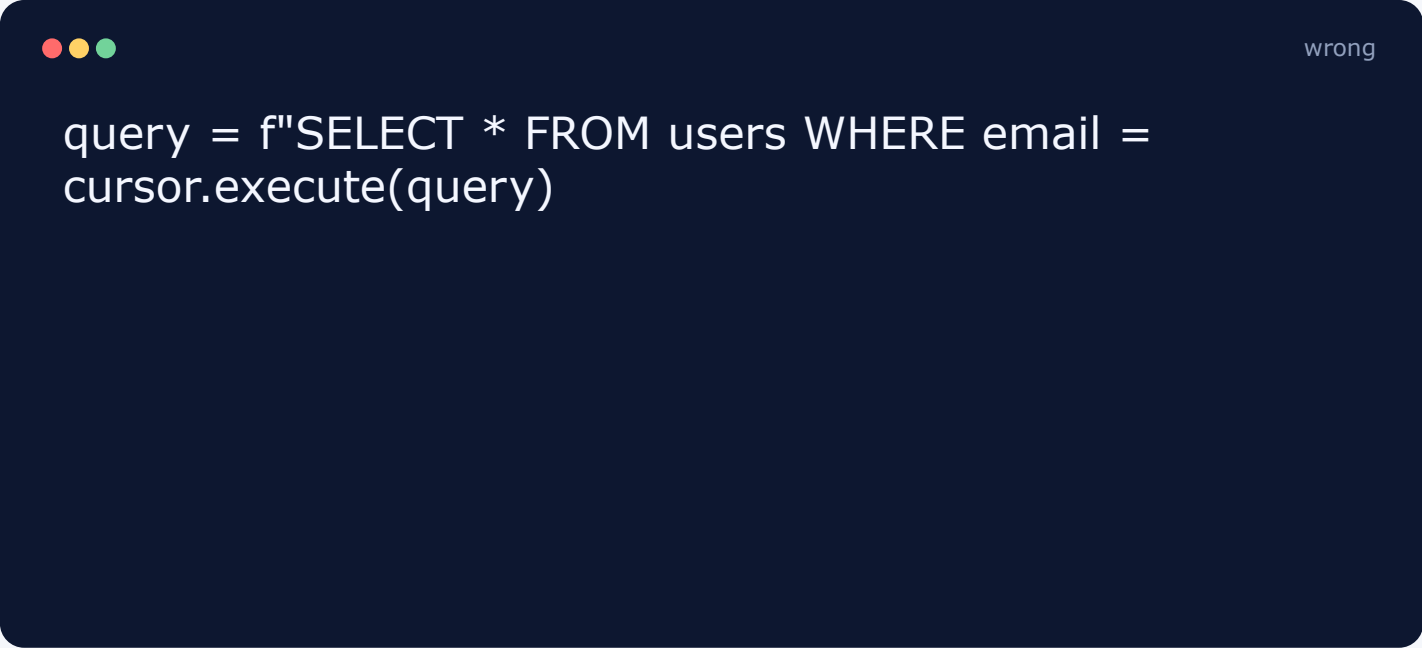
ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



```
query = f"SELECT * FROM users WHERE email =  
cursor.execute(query)
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

**Пользовательский ввод вставляется прямо в SQL-текст. Это создаёт риск SQL injection.
Нужно параметризовать запрос.**

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.

```
query = "SELECT * FROM users WHERE email = ?"  
cursor.execute(query, (email,))
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Разрешить редактирование заметки только её владельцу.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

1 Пользователь должен быть аутентифицирован.

2 Получаем `owner_id` заметки на сервере.

3 Сравниваем `owner_id` с `current_user.id`.

4 При несовпадении возвращаем 403.

5 Только затем выполняем изменение.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
def can_edit(note_owner_id, current_user_id):  
    return note_owner_id == current_user_id
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
def can_edit(note_owner_id, current_user_id):  
    return note_owner_id == current_user_id  
  
print(can_edit(7, 7))  
print(can_edit(7, 9))
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

True
False

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Какие проверки нужны перед сменой email пользователя?

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Личность, право менять именно этот аккаунт, формат/уникальность нового email, защита сессии и аудит важных изменений.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
authenticated = True  
owns_account = True  
email_is_valid = True  
if authenticated and owns_account and email_is_valid:  
    print("можно продолжить")
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

можно продолжить

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Перенеси DATABASE_URL из исходного кода в переменную окружения.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



mini_project

```
import os
DATABASE_URL = os.environ["DATABASE_URL"]
# значение не хранится в репозитории
```

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

Секрет/конфигурация отделены от исходного кода

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Хранить пароли в открытом виде.

02

Считать authentication достаточной без authorization.

03

Склеивать SQL из пользовательских строк.

04

Коммитить .env, токены и приватные ключи.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «vulnerability» и какую проблему это понятие помогает описать?

2

Чем «vulnerability» отличается от «authorization»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 vulnerability — слабое место, которое можно использовать во вред.

2 vulnerability: слабое место, которое можно использовать во вред. authorization: проверка разрешения на конкретное действие.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 Значение email передаётся отдельно от SQL-текста

4 Пользовательский ввод вставляется прямо в SQL-текст. Это создаёт риск SQL injection. Нужно параметризовать запрос.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Увидеть базовые риски веб-приложений: injection, XSS/CSRF, секреты, authentication vs authorization, hashing и принцип наименьших привилегий.
- ✓ Читать и объяснять пример темы «Безопасность приложений».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Деплой, Docker, облако и CI/CD

Завтра: Деплой, Docker, облако и CI/CD. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.